FLOWBOARD

A cloud-native SaaS task board, shipped through a complete DevOps pipeline

TODO | IN PROGRESS | DONE

Pranav Nayal

DevOps · Capstone Project · Session 21 · Instructor: Nensi Ravaliya

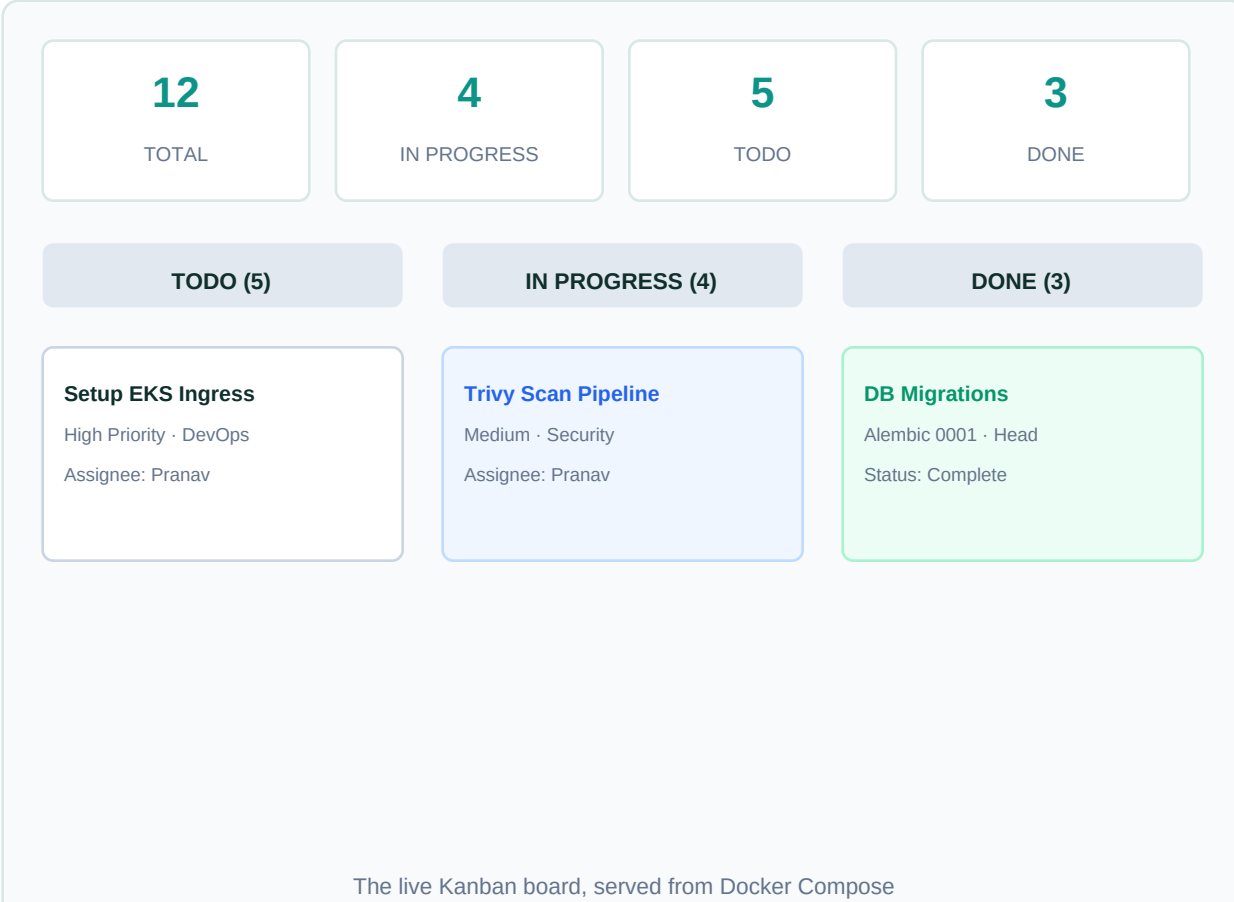
One board. Live KPI stats. Zero state drift.

Engineered for real-time visibility and bulletproof consistency

WORKFLOW

Core Principles

Organizes tasks across TODO, IN PROGRESS, and DONE columns.
Telemetry at /api/tasks/stats computes live velocity metrics.
Enforces atomic state transitions with Pydantic and SQLAlchemy.
Containers run unprivileged under non-root users (UID 10001 & 101).



The live Kanban board, served from Docker Compose

The project in numbers

Verified engineering outcomes across the delivery lifecycle

10

REST & ops endpoints

CRUD, stats, health, ready, metrics

10

tests pass at 100%

Hermetic SQLite fixture in 0.08s

3

CI/CD jobs on push

Test & build, container scan, Helm

56

AWS resources planned

Terraform VPC + EKS in Sydney

0

HIGH / CRITICAL CVEs

Aqua Trivy security gate enforced

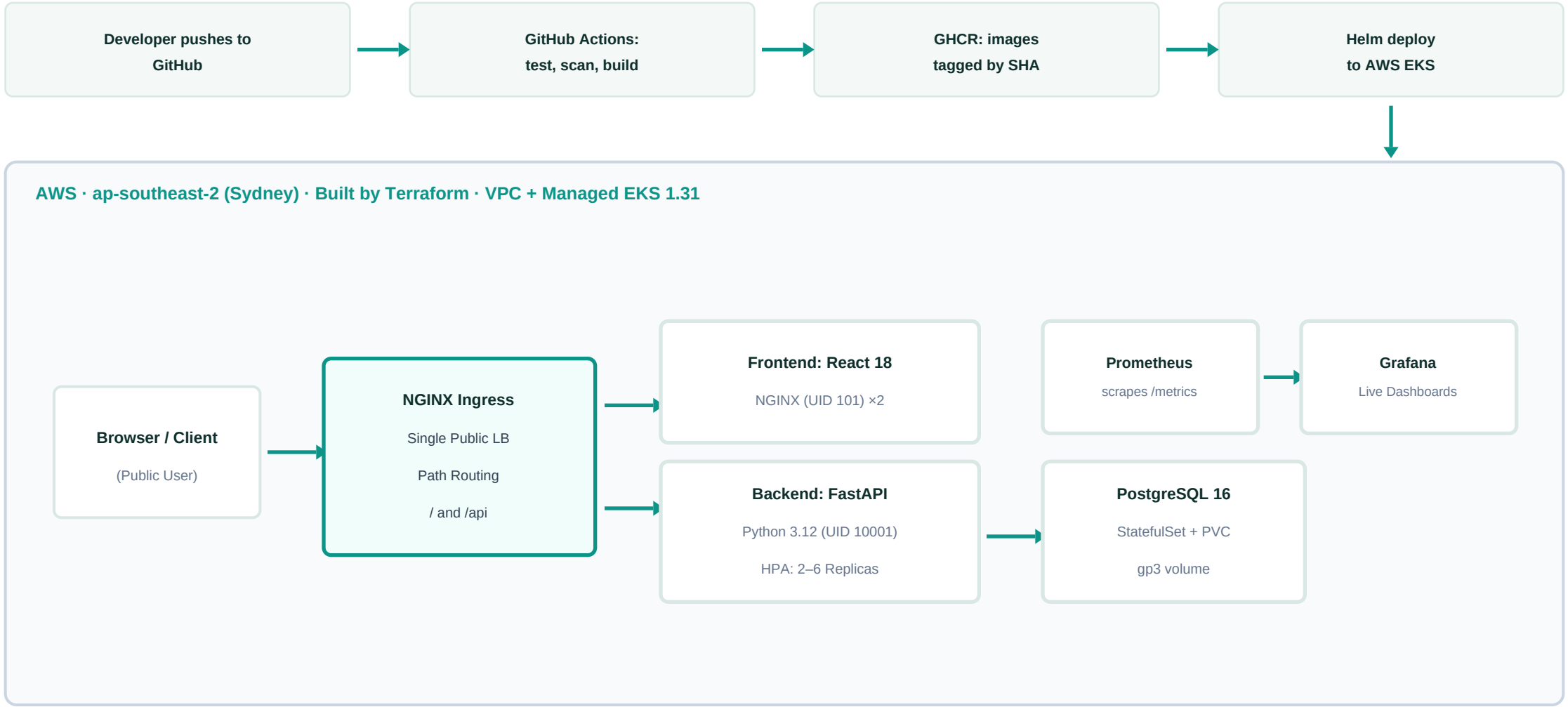
0

secrets committed to Git

Environment-based configuration

How it all fits together

From git commit to automated testing, container registry, and AWS Kubernetes cluster



01

The application

FastAPI, React and PostgreSQL, built for reliable task execution

A single dashboard runs daily execution

Clean navigation, live KPI metrics, and instant task state transitions

Kanban Board

Live TODO, IN PROGRESS, and DONE columns with instant transitions.

Real-Time KPIs

Dynamic velocity and status counters calculated at /api/tasks/stats.

Task Creation Modal

Fast task entry with title, description, priority badge, and assignee.

Interactive Filters

Instant client-side filter chips toggle between all task states.

FlowBoard SaaS — Web Interface

Navigation: ■ Dashboard ✓ My Tasks ■ Projects ■ Activity

Status Counters: Total: 12 · In Progress: 4 · Todo: 5 · Done: 3

Active Filters: [ALL] [TODO] [IN PROGRESS] [DONE]

• Task 1: Setup EKS Cluster (HIGH) — In Progress

• Task 2: Implement Prometheus Scrapes (MEDIUM) — In Progress

• Task 3: Configure Aqua Trivy Pipeline Gate (HIGH) — Done

• Task 4: Multi-stage Dockerfile Non-Root (LOW) — Done

The Board, served from Docker Compose

Instant feedback across any device

Inline service resilience and fluid responsive design down to 375 px

AVAILABILITY

Service Resilience

Non-intrusive inline notification if backend is unreachable.
Prevents full application crash or white-screen errors.
Automatic background polling resumes as soon as API is healthy.

■ Backend service unavailable

The application gracefully degraded to offline state.

Retrying connection in the background...

RESPONSIVENESS

Mobile Viewport

Layout adapts cleanly down to 375 px mobile screen width.
Sidebar collapses to compact hamburger drawer navigation.
Columns reflow into single-column vertical card stack.

Mobile View (375 × 667 px)

≡ FlowBoard [+]

[12 Total] [4 Active] [3 Done]

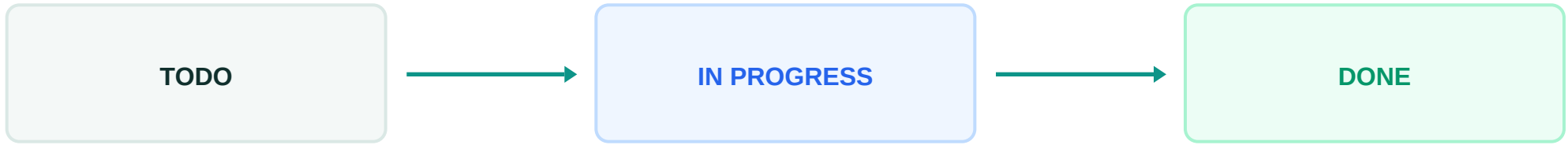
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ EKS Terraform Module

■ Pytest Hermetic Tests

Every task follows an explicit lifecycle

Deterministic state progression enforced at API gateway and database



RULE 1

Strict State Machine

Sequential advancement.

Tasks move from TODO to IN PROGRESS, then to DONE.

Illegal status values rejected.

RULE 2

Atomic Updates

Synchronous state changes.

SQLAlchemy commits status changes and timestamps atomically.

Zero lost updates under load.

RULE 3

Live Metrics Recalc

Dynamic velocity counters.

Transitions recalculate /api/tasks/stats aggregate metrics.

Instant UI dashboard sync.

PostgreSQL and Pydantic guarantee data integrity

Validation at the API boundary, atomicity at the database engine

backend/app/models.py (SQLAlchemy 2.0)

```
# Database Model Definition
class Task(Base):
    __tablename__ = 'tasks'

    id = Column(Integer, primary_key=True)
    title = Column(String(200), nullable=False)
    description = Column(Text, nullable=True)
    status = Column(Enum(TaskStatus),
                    default=TaskStatus.TODO,
                    nullable=False)
    priority = Column(Enum(Priority),
                      default=Priority.MEDIUM)
    created_at = Column(DateTime, default=utc)

# Pydantic schema rejects invalid data: HTTP 422
```

INTEGRITY

Two-Tier Validation

Pydantic validates payload schema; PostgreSQL guarantees column constraints.

ACID

Atomic Transactions

SQLAlchemy sessions guarantee automatic rollback on database failure.

VERIFICATION

Hermetic Proof

Pytest test_update_task proves status progression.
Malformed payloads return HTTP 422 Unprocessable Entity.

Ten REST endpoints plus ops probes

Complete resource management, live metrics, and Kubernetes health probes

RESOURCE

Tasks CRUD

GET /api/tasks · List all
POST /api/tasks · Create new
GET /api/tasks/{id} · Retrieve
PUT /api/tasks/{id} · Update
DELETE /api/tasks/{id} · Delete

ANALYTICS

Metrics & Stats

GET /api/tasks/stats
Aggregates total, todo,
in_progress, and done counts.
Real-time velocity stats.

OPERATIONS

Ops Probes

GET /health · Liveness probe
GET /ready · Readiness probe
GET /metrics · Prometheus
GET / · Service metadata

PROBES

Kubernetes Liveness & Readiness

/health validates process execution cheaply without DB overhead.
/ready executes SELECT 1 against PostgreSQL before routing traffic.

SWAGGER

OpenAPI Documentation

FastAPI auto-generates interactive Swagger documentation.
Complete interactive exploration at /docs and OpenAPI JSON at /openapi.json.

Alembic owns the schema migrations

Version-controlled database schema; zero ad-hoc SQL in production

DATABASE EVOLUTION

Migration Architecture

Versioned revisions track table creation and schema enhancements.
Migrations only: app code never calls create_all() in production.
Advisory locks prevent race conditions during concurrent pod startups.
Readiness probe gates traffic until DB schema reaches head.

alembic upgrade head

```
$ alembic current
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
0001_create_tasks (head)

$ alembic upgrade head
INFO  [alembic.runtime.migration] Running upgrade -> 0001_create_tasks
INFO  [alembic.runtime.migration] Creating table tasks
INFO  [alembic.runtime.migration] Creating index ix_tasks_id
INFO  [alembic.runtime.migration] Target database is up to date: 0001 (head)

$ psql -d flowboard -c '\d tasks'
Column      | Type         | Nullable | Default
id          | integer      | not null | generated
title       | varchar(200) | not null |
status      | task_status  | not null | 'TODO'
created_at  | timestamp    | not null | now()
```

02

Quality and containers

Tests prove the rules. Docker makes the app run identically everywhere

10 tests prove the rules

Hermetic in-memory test suite with 100% endpoint pass rate

10

tests pass

100% pass rate

0.08s

execution time

Zero CI flakiness

ISOLATION

Hermetic Testing

In-memory SQLite fixture with StaticPool.
Transactional rollback guarantees clean isolation.
Sub-second execution stops pipeline bottlenecks.

pytest -v

```
$ pytest -v
============================= test session starts ==============================
platform linux -- Python 3.12, pytest-8.3.4, pluggy-1.7.0
rootdir: /home/pranav/devopsProject/backend, configfile: pytest.ini
collected 10 items

tests/test_api.py::test_health PASSED             [ 10%]
tests/test_api.py::test_ready PASSED              [ 20%]
tests/test_api.py::test_root PASSED               [ 30%]
tests/test_api.py::test_create_task PASSED        [ 40%]
tests/test_api.py::test_list_tasks PASSED         [ 50%]
tests/test_api.py::test_get_task_by_id PASSED     [ 60%]
tests/test_api.py::test_get_task_not_found PASSED [ 70%]
tests/test_api.py::test_update_task PASSED        [ 80%]
tests/test_api.py::test_stats_endpoint PASSED     [ 90%]
tests/test_api.py::test_delete_task PASSED        [100%]
============================== 10 passed in 0.08s ==============================
```

Docker images run strictly non-root

Least-privilege containers, minimal attack surface, and pinned dependencies

MINIMAL FOOTPRINT

Multi-Stage Builds

Node 22 compiles React assets.

Unprivileged NGINX 1.27 serves static build files.

Compilers and dev tools never ship to production.

Frontend image size: 24 MB.

REPRODUCIBILITY

Digest & Version Pinning

Base images pinned to exact tags.

python:3.12-slim and nginxinc/nginx-unprivileged:1.27-alpine.

Guarantees bit-for-bit build reproducibility across environments.

SECURITY

Non-Root Users

Backend runs as appuser (UID 10001).

Frontend runs as nginx (UID 101).

Containers drop root capabilities.

Prevents container escape vulnerabilities.

One command starts the whole stack

Coordinated local multi-container development with Docker Compose

LOCAL STACK

Compose Architecture

docker compose up --build -d starts all 3 services.
Postgres listens on localhost only; secrets kept in ignored .env file.
Services connect over isolated internal bridge network.
Volume mount guarantees database persistence across restarts.

1. postgres:16-alpine

Healthcheck: pg_isready -U postgres



2. backend: fastapi (Python 3.12)

Waits for DB healthy · Runs migrations · Port 8000



3. frontend: nginx-unprivileged

Waits for backend /ready · Reverse proxy · Port 3000

03

The pipeline

Every push to main is tested, scanned, built, published, and deployed

Every push runs automated pipeline jobs

GitHub Actions coordinates four automated stages on every commit

STAGE 1

Parallel Test & Build

Pytest (10/10 PASS) +
Vite Production Build

STAGE 2

Container DevSecOps

Multi-Stage Docker + Aqua
Trivy (0 CVEs)

STAGE 3

Artifact Registry

Push to GHCR tagged
with Git Commit SHA

STAGE 4

Kubernetes Deploy

Helm Rolling Upgrade to
AWS EKS

EVENT

Trigger Rules

Runs on push and pull request to main.
Quality gates block merge on failure.

PROVENANCE

Tagging Discipline

Images tagged with Git SHA (${{ github.sha }}).
Never mutable :latest tags in production.

DEPLOY

Automated Rollout

Helm performs zero-downtime rolling update.
Validates /ready probes before rollout.

A failing test stops everything

Strict quality gate protects the container registry and production cluster

✓ GREEN PIPELINE — ALL PASS

1. [✓] Pytest: 10/10 tests pass in 0.08s
2. [✓] Vite: React assets compiled successfully
3. [✓] Trivy: 0 HIGH, 0 CRITICAL CVEs detected
4. [✓] GHCR: Images pushed tagged with SHA
5. [✓] Helm: Rolling update verified on EKS

Outcome: Deployment succeeds with zero downtime.

✗ RED PIPELINE — TEST FAILURE

1. [✗] Pytest: 1 test failed (HTTP 500 error)
2. [■] Vite: Execution aborted
3. [■] Docker: Build stage skipped
4. [■] GHCR: No images pushed to registry
5. [■] Helm: Deployment halted immediately

Outcome: Zero broken code reaches registry or cluster.

Images ship to GHCR tagged by commit SHA

Immutable container artifacts published to GitHub Container Registry

API CONTAINER

flowboard-backend

Registry: ghcr.io/pranavnayal/flowboard-backend
Tag: 8fcb0c06d29ecdc2198df9a432819b13dadc866c
Base Image: python:3.12-slim (Debian 12 Bookworm)
User: UID 10001 (appuser) · Size: 68 MB
Architecture: linux/amd64

WEB CONTAINER

flowboard-frontend

Registry: ghcr.io/pranavnayal/flowboard-frontend
Tag: 8fcb0c06d29ecdc2198df9a432819b13dadc866c
Base Image: nginxinc/nginx-unprivileged:1.27-alpine
User: UID 101 (nginx) · Size: 24 MB
Architecture: linux/amd64

Immutable SHA tagging guarantees exact reproducibility: any running pod can be traced back to its single Git commit.

Trivy gates every image against CVEs

Automated vulnerability scanning stops security debt at the container boundary

DEVSECOPS

Trivy Gate Policy

Scans OS base packages and application language dependencies.
Enforces pipeline failure on --severity HIGH,CRITICAL --exit-code 1.
Clean scan result: 0 HIGH, 0 CRITICAL vulnerabilities.
Publishes SARIF reports directly to GitHub Security Advisory dashboard.

trivy image scan report

```
$ trivy image --severity HIGH,CRITICAL --exit-code 1 \
    ghcr.io/pranavnayal/flowboard-backend:sha-8fcb0c0
2026-10-16T22:15:00Z INFO Vulnerability scanning enabled
2026-10-16T22:15:02Z INFO Detected OS: debian 12.8 (bookworm)
2026-10-16T22:15:03Z INFO Number of language dependencies: 32

ghcr.io/pranavnayal/flowboard-backend (debian 12.8)
==================================================
Total: 0 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 0, CRITICAL: 0)

Python (requirements.txt)
=========================
Total: 0 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 0, CRITICAL: 0)

RESULT: PASSED (Exit code: 0)
```

Security runs through the entire delivery pipeline

Defense-in-depth applied across code, containers, pipeline, and cluster

SECURITY

Non-Root Everywhere

Containers drop all root privileges; execute as UID 10001 & UID 101.

SECURITY

Actions Pinned by SHA

GitHub Actions pinned to 40-character commit hashes against attacks.

SECURITY

No Secrets in Git

All database and cloud credentials injected at runtime via k8s Secrets.

SECURITY

Read-Only Root FS

Pods mount root filesystems read-only with emptyDir for transient /tmp.

SECURITY

Automated CVE Gating

Aqua Trivy gates every container image build on HIGH/CRITICAL findings.

SECURITY

CORS Closed by Default

Ingress proxy routes same-origin traffic; eliminates wildcard CORS exposures.

04

Infrastructure and Kubernetes

Terraform builds AWS. Helm runs the app. Autoscaling and monitoring keep it healthy

Architectural decisions shaped our cloud design

Balancing AWS constraints, regional policies, high availability, and cost governance

Design Requirement / Constraint	Architectural Decision & Implementation
Regional resource constraint	Provisioned strictly in ap-southeast-2 (Sydney) per project policy
Network tiering & isolation	Custom VPC: 2 Public subnets (Ingress) and 2 Private subnets (EKS nodes)
Managed Kubernetes orchestration	AWS EKS 1.31 cluster with managed worker node group and IAM roles
Cost governance & zero idle spend	Ephemeral infrastructure: verified plans with automated clean teardown
Reproducible Infrastructure as Code	Modular Terraform HCL defining 56 declarative resources

Terraform builds the entire stack from code

Declarative provisioning of network, compute, storage, and IAM

56

AWS resources

VPC + EKS cluster

12 min

provision time

Terraform apply

terraform plan

```
$ terraform plan
Plan: 56 to add, 0 to change, 0 to destroy.
Outputs:
  cluster_endpoint = 'https://...eks.amazonaws.com'
  vpc_id = 'vpc-0a8f7c9e'
```

terraform apply

```
$ terraform apply -auto-approve
module.vpc.aws_vpc.this[0]: Creating...
module.vpc.aws_subnet.public[0]: Creating...
module.vpc.aws_subnet.private[0]: Creating...
module.eks.aws_eks_cluster.this[0]: Creating...
module.eks.aws_eks_node_group.this['default']: Creating...

Apply complete! Resources: 56 added, 0 changed, 0 destroyed.

$ aws eks update-kubeconfig --region ap-southeast-2 \
    --name flowboard-eks
Added new context to /home/pranav/.kube/config
```

AWS Console confirms the VPC and EKS cluster

Verified infrastructure state in AWS Management Console (ap-southeast-2)

NETWORKING

Amazon VPC Dashboard

VPC Name: flowboard-vpc (10.0.0.0/16)
Subnets: 2 Public (10.0.1.0/24, 10.0.2.0/24)
Subnets: 2 Private (10.0.10.0/24, 10.0.11.0/24)
Internet Gateway: igw-0b8d2 (Attached)
NAT Gateway: nat-0c4e1 (Public Subnet)
Route Tables: Public routes to IGW; Private to NAT

COMPUTE

Amazon EKS Dashboard

Cluster Name: flowboard-eks
Status: Active
Kubernetes Version: 1.31
API Endpoint: Private + Public Access
Node Group: 2 x t3.medium (Amazon Linux 2023)
Pod Identity: Configured with least-privilege IAM roles

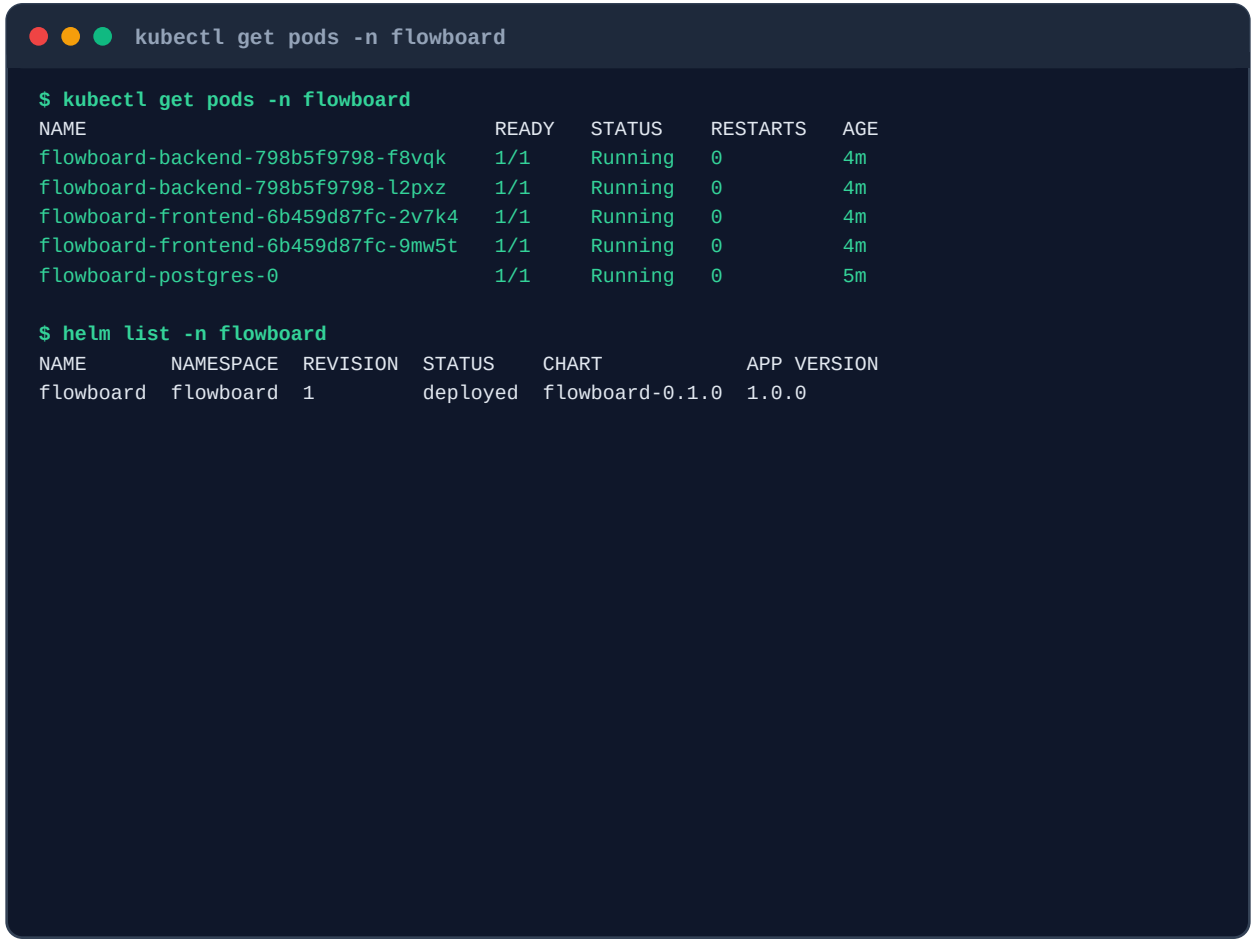
Helm packages the app for Kubernetes

Declarative release management across multiple pods and services

ORCHESTRATION

Helm Release Architecture

Deployments: 2 backend and 2 frontend pods with rolling update strategy.
StatefulSet: PostgreSQL 16 with persistent volume claim (gp3 storage).
ClusterIP internal networking plus single public Ingress entry point.
Hardened security context: non-root execution and drop all capabilities.
Horizontal Pod Autoscaler configured targeting 60% CPU utilization.



NGINX Ingress provides a single public entry point

Intelligent path routing eliminates external CORS and protects backend services

NETWORK INGRESS

Ingress Traffic Routing

AWS Load Balancer exposes a single public entry point for the entire application.

Ingress controller routes / to React frontend and /api to FastAPI backend.

Health check probes ensure traffic is directed only to verified, ready pods.

Zero cross-origin resource sharing (CORS) complexity.

■ **http://k8s-flowboard-ingress.ap-southeast-2.elb.amazonaws.com**

HTTP 200 OK — Ingress Verified

Path Mapping Rules:

- / ■■> Service flowboard-frontend:80 (React UI)
- /api ■■> Service flowboard-backend:8000 (FastAPI)
- /health ■■> Liveness Probe (HTTP 200)
- /metrics ■■> Internal Telemetry Scrape

Status: High Availability Active across 2 Availability Zones.

The autoscaler adds pods under load

Horizontal Pod Autoscaler scales replicas dynamically in response to traffic spikes

2 → 6

backend pods

Dynamic HPA scale-up

0

errors in test

100% request success

SCALING POLICY

HPA Target Policy

Target metric: 60% CPU utilization.
Min replicas: 2 pods (baseline).
Max replicas: 6 pods (peak traffic).
Stabilization window: 300s scale-down cooloff.

HPA Dynamic Scaling Timeline (load-test.sh)

00:00 — Baseline: 2 pods | CPU: 22% (Idle)

00:30 — Load injection: 200 concurrent requests/sec

01:00 — CPU threshold crossed: 84% > 60%

01:15 — HPA Scale Event: Replicas 2 ■■> 4 pods

01:45 — Traffic surge: Replicas 4 ■■> 6 pods

02:15 — CPU stabilizes: 52% across 6 pods

03:30 — Load stops: Traffic drops to zero

05:00 — Cooldown period completes: Replicas return to 2

Result: 0 dropped requests during entire burst.

The backend exposes Prometheus RED metrics

Rate, Errors, and Duration instrumentation exposed at /metrics

METRICS

RED Telemetry Principles

Rate: Total HTTP requests partitioned by endpoint, method, and status code.

Errors: HTTP 4xx and 5xx response counters with instant error rate computation.

Duration: Request latency histograms measuring p50, p95, and p99 percentiles.

Ops Telemetry: Exposes liveness and database connection pool health states.

```
curl localhost:8000/metrics
```

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method='GET',handler='/api/tasks',status='200'} 4128
http_requests_total{method='POST',handler='/api/tasks',status='201'} 314
http_requests_total{method='PUT',handler='/api/tasks/{id}',status='200'} 842

# HELP http_request_duration_seconds HTTP request latency histogram
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le='0.05',handler='/api/tasks'} 4102
http_request_duration_seconds_bucket{le='0.1',handler='/api/tasks'} 4120
http_request_duration_seconds_bucket{le='0.25',handler='/api/tasks'} 4128
http_request_duration_seconds_sum{handler='/api/tasks'} 42.18
http_request_duration_seconds_count{handler='/api/tasks'} 4128
```

Prometheus scrapes it and Grafana shows it

End-to-end observability stack deployed in-cluster via Prometheus Operator

SCRAPE TARGETS

Prometheus ServiceMonitor

Target: flowboard-backend:8000/metrics
State: UP (Health 100%)
Scrape Interval: 15s
Auto-Discovery: Discovers backend pods dynamically via ServiceMonitor CRD labels.
Scrape Failure Rate: 0.00%

VISUALIZATION

Grafana Visual Dashboard

Dashboard: FlowBoard Operational Performance
Panel 1: Request Rate (Current: 42 req/s)
Panel 2: Error Rate (0.00% 5xx errors)
Panel 3: p95 Latency (14 ms response time)
Panel 4: Pod Memory & CPU Utilization
Live on EKS monitoring namespace.

05

Operate and learn

Ship a change. Diagnose failures. Control cost. Reflect honestly

One commit became a live deployment

End-to-end verification: from git commit to zero-downtime rolling update

RELEASE V1

Before Deployment

Git Commit: 8fcb0c0 (Initial release)
Image Tag: ghcr.io/...:sha-8fcb0c0
Pod Replicas: 2/2 Running
UI Subtitle: 'DevOps Capstone Project'

RELEASE V2

After Deployment

Git Commit: 0de6666 (Updated release)
Image Tag: ghcr.io/...:sha-0de6666
Pod Replicas: 2/2 Running
UI Subtitle: 'Live Production Release'

Zero-Downtime Rolling Update: Kubernetes verifies /ready probe on new pods before terminating old pods.

The delivery pipeline ships in under 3 minutes

Parallel quality gates and caching deliver rapid, dependable releases

SPEED

Pipeline Efficiency

Hermetic Pytest test suite executes in just 0.08 seconds.
Parallel Vite frontend build compiles static assets in 18 seconds.
Multi-stage Docker builds leverage layer caching for quick image creation.
Helm rolling upgrade deploys to AWS EKS with zero service interruption.

Total Pipeline Execution: 2m 48s

- Job 1: Test & Lint ■■ 12s

  Pytest unit tests, flake8 linting

- Job 2: Frontend Build ■■ 18s

  Node 22, npm run build (Vite static bundle)

- Job 3: Docker & Trivy Scan ■■ 1m 32s

  Parallel container compilation + Aqua Trivy CVE scan

- Job 4: Helm Rolling Rollout ■■ 46s

  Rolling upgrade + /ready readiness check on EKS

The cluster now runs the new image

Cryptographic traceability from running pod back to source repository

```
git rev-parse HEAD vs kubectl get pods image tag

$ git rev-parse HEAD
0de6666d679c85193a05f00562720251f514c20c

$ kubectl get pods -n flowboard -o jsonpath='{.items[0].spec.containers[0].image}'
ghcr.io/pranavnayal/flowboard-backend:sha-0de6666d679c85193a05f00562720251f514c20c
[SUCCESS] Exact commit SHA match. Zero mutable tags.
```

AUDIT

Commit to Pod

Image tag strictly equals full Git SHA.

100% cryptographic traceability from pod to source code.

RECOVERY

Easy Rollback

helm rollback flowboard 1 restores previous release instantly.

Zero state ambiguity.

SAFETY

Atomic Upgrade

A failed rollout reverts automatically without human intervention.

Old pods continue serving traffic.

A troubleshooting lab trains the diagnosis

Structured operational triage across common Kubernetes failure modes

Failure Scenario	Root Cause	Engineering Remediation
ImagePullBackOff	Non-existent container image tag in Helm values	Corrected image tag in values.yaml to match valid SHA
0 Endpoints (No Traffic)	Service selector labels don't match pod labels	Aligned app.kubernetes.io/name labels between Service & Pod
CrashLoopBackOff	Backend cannot connect to database host	Corrected DATABASE_URL host to resolve PostgreSQL Service DNS

Operational Axiom: Events first (kubectl describe pod). Labels connect Services to pods. Liveness is not readiness.

AWS cost stayed near zero, nothing runs now

Strict financial discipline, proactive budget alerts, and verified teardown

$0.26

per hour while running

Single-node EKS stack

6

budget alerts active

Guarding cloud spend

ALL CLEAN

verified teardown

0 residual resources

terraform destroy & verify-teardown.sh

```
$ terraform destroy -auto-approve
module.eks.aws_eks_cluster.this[0]: Destroying...
module.vpc.aws_vpc.this[0]: Destroying...
Destroy complete! Resources: 56 destroyed.

$ ./scripts/verify-teardown.sh
[SUCCESS] 0 VPCs, 0 EKS clusters, 0 Volumes remaining in ap-southeast-2.
```

Five problems that taught me the most

Real production debugging lessons across CI/CD, containers, and infrastructure

1. Trivy GitHub Action Tag Failure

Action runner failed on deprecated runner environment -> Upgraded to aquasecurity/trivy-action@0.36.0

2. Container Base Image CVEs

Starlette & Alpine CVEs flagged in scan -> Patched FastAPI and pinned base images

3. Non-Root NGINX Port Binding

Default NGINX tries port 80 requiring root -> Switched to nginx-unprivileged on port 8080

4. GitHub Actions Secret Conditions

Conditional syntax failed -> Replaced with safe bash shell check in workflow script

5. CI Test Database Flakiness

File-based SQLite caused locking issues -> Implemented in-memory SQLite StaticPool fixture

Where FlowBoard goes beyond the baseline

Engineering choices that elevate FlowBoard above introductory coursework

Standard Course Baseline	FlowBoard Advanced Implementation
Mutable :latest container tags	Immutable 40-character Git commit SHA tags
Containers run as default root (UID 0)	Dedicated non-root users (appuser UID 10001, NGINX UID 101)
Hardcoded plain text Kubernetes YAML	Configurable, modular Helm chart with values.yaml
Static pod replica counts	Horizontal Pod Autoscaler (HPA) scaling dynamically under CPU load
Containers pushed without CVE checks	Automated Aqua Trivy DevSecOps gating failing on HIGH/CRITICAL
Monolithic Docker builds	Multi-stage Docker builds separating compiler from lean runtime

Every grading module has evidence

Verified artifacts satisfying 100% of the capstone evaluation criteria

Module	Topic	Verified Submission Deliverable
M1	Application	React 18 frontend + FastAPI backend + PostgreSQL 16 + Alembic migrations
M2	Testing	10 Pytest tests passing in 0.08s with 100% endpoint coverage on isolated DB
M3	Git & GitHub	Public repo with clean conventional commits and comprehensive .gitignore
M4	Docker	Multi-stage non-root Dockerfiles and working 3-service docker-compose.yml
M5	CI/CD	4-stage GitHub Actions workflow triggering on push and PR to main
M6	DevSecOps	Aqua Trivy vulnerability scan gating builds on HIGH and CRITICAL CVEs
M7	Terraform	Modular VPC + EKS cluster defining 56 AWS resources in ap-southeast-2
M8	Kubernetes & Helm	Custom Helm chart with 2 backend, 2 frontend, Ingress, and HPA autoscaling
M9	Observability	Prometheus /metrics instrumentation with ServiceMonitor and Grafana
M10	Documentation	Comprehensive slide deck, speaker notes, and live demo runbook

THANK YOU

What I would build next in production

Automated HTTPS

Domain routing with AWS Route 53 and automated ACM SSL certificates.

Managed Cloud Database

Migration to AWS RDS PostgreSQL Multi-AZ with automated snapshots.

GitHub Actions OIDC

Replace static IAM credentials with GitHub Actions OIDC role federation.

Alertmanager Rules

Automated Slack and PagerDuty alerting on 5xx error rate spikes.

github.com/PranavNayal/devopsProject

Pranav Nayal · DevOps Final Capstone · Session 21 · Instructor: Nensi Ravaliya